

Julia

Relatório Técnico — Arquitetura, Funcionalidades e Regras de Negócio

Documento gerado a partir de estudo profundo do código-fonte
c:\dev\appjulia · 2026-07-17

Sumário

1. [Visão Geral do Projeto](#)
2. [Arquitetura & Camada de Dados](#)
3. [Módulo Chat / WhatsApp](#)
4. [Integração UaZapi — Detalhe Técnico](#)
5. [CRM, Agentes de IA, Followup, Contratos, Jurídico](#)
6. [Tickets, Telemetria, Notificações](#)
7. [Telefonia, Pagamentos, Vídeo](#)

Visão Geral do Projeto

SaaS multi-tenant para escritórios de advocacia: atendimento omnichannel via WhatsApp (com agentes de IA), CRM, helpdesk, telefonia, telemetria e integrações jurídicas. Deploy via Lovable Cloud.

Esta documentação foi gerada por um estudo profundo do código (não é aspiracional — reflete o que existe). Ela é o ponto de entrada; os detalhes por domínio estão em `docs/`.

Stack

- **Frontend:** React 18 + Vite 5 + TypeScript (config frouxo: `strictNullChecks:false`, `noImplicitAny:false`) + shadcn/Radix + Tailwind + React Query (`staleTime 30s`, `refetchOnWindowFocus:false`) + React Router 6. Alias `@/*` → `./src/*`.
- **Backend:** Supabase (Postgres gerenciado + ~118 edge functions Deno + Storage + Realtime), **mais um Postgres externo legado** (managed DB, porta 25061) acessado só via a edge function `db-query`.
- **Auth:** própria (bcrypt), não usa Supabase Auth. Sessão em `localStorage`.
- **Telefonia:** SIP (api4com, 3cplus) via JsSIP + Wavoip (chamadas por WhatsApp) + vellip (CDR only).
- **Vídeo:** Daily.co (`@daily-co/daily-js`).
- **Pagamentos:** Mercado Pago, Asaas, InfinityPay.
- **IA:** Lovable AI Gateway (Gemini 2.5 Flash / 3 Flash Preview por padrão) com fallback OpenRouter, configurável por feature em `client_ai_model_config`.

Os Dois Bancos de Dados

1. **Supabase** — tabelas app-native: chat (maior domínio, ~50 tabelas), CRM Builder, tickets, telemetria, telefonia (Wavoip/SIP), billing, ajuda, notificações. Cliente: `src/integrations/supabase/client.ts`.
2. **Postgres externo (legado)** — `users`, `clients` (= tenant real), `agents` (agente de IA, `cod_agent`), `user_agents`, `sessions`, `log_messages`, `crm_atendimento_cards/stages/history` (CRM de leads "clássico"), `campaing_ads` (Meta Ads), `modules / user_permissions`, tabelas Advbox. Acesso **exclusivo** via edge function `db-query` (`supabase/functions/db-query/index.ts`, ~120 actions) e o wrapper `src/lib/externalDb.ts`.

Detalhe completo (todas as actions do `db-query`, schema por domínio): [docs/data-layer.md](#).

⚠ Postura de Segurança (ler antes de mexer em auth/dados)

- **RLS do Supabase é permissiva em quase tudo:** `USING (true) WITH CHECK (true)` confirmado em ~100 migrations. Isolamento de tenant **não é garantido pelo banco**, só pela aplicação.
- **db-query expõe a action raw:** executa `sql.unsafe(query, params)` com a query vinda do frontend (`externalDb.raw(...)`), sem allowlist de tabela/verbo. A action `select` genérica também interpola coluna/orderBy sem sanitização.
- **40+ edge functions com `verify_jwt=false`** (`supabase/config.toml`) — esperado para webhooks de terceiros (Meta, Asaas, uazapi, api4com, 3cplus, Wavoip, InfinityPay), mas algumas não-webhook também estão abertas (`waba-send`, `send-push`, `ai-provider-key-set`, `queue-*`, `telemetry`).
- **Multi-tenancy por `client_id`** é resolvido em runtime (`resolveEffectiveClientId`, herança via `users.user_id` → `parent.client_id`) e filtrado no código/queries — não há constraint de banco impedindo cross-tenant.
- Rotas `/chat/*` (24 sub-rotas) e `/advbox/*` **não têm proteção de módulo** no `ProtectedRoute` — só exigem estar autenticado.

Isso não é bloqueante para features novas, mas qualquer mudança em auth, RLS, ou na função `db-query` deve ser tratada como sensível (staging + incremental, nunca big-bang).

Autenticação & Autorização

- Login: `externalDb.login(email, password)` → bcrypt verificado no `db-query` (nunca no browser). `is_active=false` bloqueia.
- Roles (`AppRole`): `admin` | `user` | `colaborador` | `time` | `advogado` | `comercial`. `admin` sempre passa em `hasPermission`. `time/advogado/comercial` são membros de equipe vinculados a um titular (`user / admin`) e herdam `client_id`.

- Permissões por módulo (`ModuleCode` U string dinâmica para embeds): `can_view/create/edit/delete` , custom por usuário (`use_custom_permissions`) ou herdado de `role_default_permissions` .
- Inatividade: `INACTIVITY_TIMEOUT_MS = 30min` (const real; comentários no código dizem "1h" — desatualizado). Logout sincroniza entre abas via `storage` event.
- Avatar: prioridade `user_avatars.photo_url` (Supabase, por usuário) → fallback `clients.photo` (externo, logo do escritório).

Detalhe completo: [docs/data-layer.md](#).

Versionamento

Build-time auto-bump de PATCH (`vite-plugin-auto-version.ts`): `MAJOR.MINOR` = maior entre `package.json` e `public/version.json` ; PATCH incrementa a cada build. No login, `checkVersionAndReloadIfNeeded` compara `/version.json` com `__APP_VERSION__` embutido; se diferente, desregistra Service Workers, limpa caches, preserva só `AUTH_USER` / `AUTH_LAST_ACTIVITY` e recarrega.

Módulos (domínios funcionais)

Domínio	Rota raiz	Módulo (<code>ProtectedRoute</code>)	Doc
Chat/Inbox omnichannel (uazapi, WABA, Instagram, WebChat)	<code>/chat/*</code> (24 sub-rotas)	nenhuma proteção de módulo	docs/chat.md
CRM de leads (WhatsApp)	<code>/crm/leads</code>	<code>crm_leads</code>	docs/crm-agents-legal.md
CRM Builder (Kanban)	<code>/crm-builder</code>	<code>crm_leads</code>	docs/crm-agents-legal.md
Agentes de IA	<code>/agente/meus-agentes</code>	<code>agent_management</code>	docs/crm-agents-legal.md
Filas (queues)	<code>/agente/filas</code>	<code>filas</code>	docs/crm-agents-legal.md
Followup	<code>/agente/followup</code>	<code>followup</code>	docs/crm-agents-legal.md
Contratos	<code>/estrategico/contratos</code>	<code>strategic_contract</code>	docs/crm-agents-legal.md
Advbox / DataJud	<code>/advbox/*</code> , <code>/datajud</code>	sem proteção / <code>datajud</code>	docs/crm-agents-legal.md
Copiloto IA / Prompts	<code>/admin/copiloto</code> , <code>/admin/prompts</code>	<code>copilot_admin</code> , <code>prompt_generator</code>	docs/crm-agents-legal.md
Tickets / Helpdesk	<code>/tickets</code>	<code>support_tickets</code>	docs/tickets-telemetry.md
Telemetria (Ambiente & Performance)	<code>/admin/monitoramento</code>	<code>admin_agents</code>	docs/tickets-telemetry.md
Notificações internas	<code>/notificar-clientes</code>	<code>notify_customers</code>	docs/tickets-telemetry.md
Telefonia (SIP + Wavoip)	<code>/telefonica</code> , <code>/wavoip</code>	<code>telephony</code> , <code>wavoip</code>	docs/telephony-payments.md
Pagamentos/Checkout	<code>/comprar</code> , <code>/admin/pedidos</code>	<code>julia_orders</code>	docs/telephony-payments.md
Vídeo (Daily.co)	<code>/video/queue</code> , <code>/call/:roomName</code>	—	docs/telephony-payments.md
Admin (permissões, módulos, planos)	<code>/admin/*</code>	<code>admin_agents</code> (maioria)	docs/crm-agents-legal.md

Tabela completa de rotas: [docs/data-layer.md](#).

Detalhe interno do webhook uazapi (parsing, dedup, pipeline de histórico): [docs/uazapi-integration.md](#).

Padrão "auto-registro de módulo"

Dezenas de hooks `useEnsure*Module.ts` (ex.: `useEnsureTicketsModule`, `useEnsureDataJudModule`) — cada módulo opcional, ao montar pela 1ª vez, garante sua própria entrada em `modules` (Postgres externo, via `externalDb.createModule`), aparecendo automaticamente no menu e na matriz de permissões sem migration manual. Reuse esse padrão para módulos novos.

Notas de leitura

- Este documento e os arquivos em `docs/` foram gerados por análise de código em 2026-06 e refletem o estado do repo naquele momento. Para features específicas recém-alteradas, prefira ler o código atual — schema evolui por migrations incrementais (`ALTER TABLE ADD COLUMN IF NOT EXISTS`), então colunas usadas no frontend podem não aparecer no `CREATE TABLE` original citado nos docs.
- Convenção de edge function: JSON `{action, data}` roteado por `switch`, service-role key, CORS aberto. Ver exemplos em `db-query`, `telemetry`, `client-files` (futuro).

Arquitetura & Camada de Dados

Stack e Build

Meta (package.json): version: 1.2.17 (auto-bumped no build).

Scripts: dev, build, build:dev, lint, preview, test (vitest), test:watch.

Dependências-chave: react@18.3, react-router-dom@6.30, vite@5.4 + @vitejs/plugin-react-swc, @tanstack/react-query@5.83, @supabase/supabase-js@2.91, jotai@2.17, shadcn/Radix, tailwindcss@3.4, recharts, @dnd-kit/* (kanban CRM), react-hook-form@7.61 + zod@3.25, @tiptap/*@3.26 (editor rico), bcryptjs@3.0.3, @daily-co/daily-js / daily-react (vídeo), @wavoip/wavoip-api / wavoip-webphone, jssip@3.10 (SIP), jspdf + xlsx (exports), vitest@3.2.

tsconfig: frouxo (noImplicitAny:false, strictNullChecks:false, skipLibCheck:true). Alias @/* → ./src/*.

vite.config.ts: porta 8080, versionFilePlugin custom grava dist/version.json, define.__APP_VERSION__, chama autoBumpVersion() no build.

Os Dois Bancos de Dados

Supabase (Postgres gerenciado — "Lovable Cloud")

src/integrations/supabase/client.ts: createClient<Database> com auth.storage=localStorage. **A auth nativa do Supabase Auth NÃO é usada** para login (login é via Postgres externo + bcrypt); o client Supabase serve para tabelas app-native, Storage, Realtime e edge functions. project_id = zenizgyrwlonmufxnjqt.

Schema (~177 tabelas/views em public), principais por domínio:

- **Chat/Inbox** (maior domínio, ~50 tabelas): chat_conversations, chat_messages, chat_contacts, chat_conversation_participants/presence/tags/summaries/history, chat_message_reactions, chat_mentions, chat_departments, chat_agent_capacity, chat_routing_rules, chat_saved_views, chat_bots / _flows / _flow_runs, chat_automation_rules / _logs, chat_campaigns / _recipients / _variants / _schedules, chat_ai_autoreply_rules / _logs / chat_ai_classifications, chat_kb_articles / _categories, chat_csat_config / _responses, chat_sla_configs, chat_api_keys, chat_webhooks / _deliveries, chat_crm_links, chat_client_settings, chat_role_permissions, chat_audit_log, chat_lgpd_requests, chat_scheduled_messages, chat_dropped_messages, chat_return_chat_runs, chat_bulk_close_logs, chat_analytics_daily, webchat_config / _sessions.
- **CRM (app-native)**: crm_boards, crm_pipelines, crm_deals / _deal_history, crm_checklists / _items, crm_custom_fields, crm_internal_notes, crm_automation_rules / _logs, crm_audit_log, crm_copilot_config / _settings / _insights / _chat_messages. CRM comercial: crm_comercial_cards / _stages / _history.
- **Tickets/Support**: support_tickets / _ticket_messages / _ticket_counters / _protocol_counters, support_categories / _departments / _team_members / _settings, support_assistant_config, support_monitored_groups / _group_messages.
- **Tarefas/Gamificação**: tasks / task_items / task_categories / task_templates / _points_ledger.
- **Telemetria/Presença**: user_activity_log, user_device_log, user_performance_log, user_presence / _daily / _heartbeats (particionada por mês), mv_user_chat_daily, mv_user_phone_daily, mv_user_sessions_daily, user_presence_status, user_last_activity, user_device_latest.
- **Telefonia**: phone_call_logs, phone_config, phone_extensions / _extension_plans / _user_plans (api4com); vellip_call_logs; Wavoip: wavoip_devices / _device_members / _device_queues / _call_logs / _providers / _plans / _orders / _user_plans / _reconcile_queue; telephony_orders / _providers.
- **Filas**: queues, queue_agent_links, queue_providers / _plans / _orders / _user_plans; view active_queues.
- **Vídeo (Daily.co)**: video_call_records, video_orders / _plans / _user_plans.
- **WhatsApp/uazapi (histórico)**: uazapi_history_items / _runs, whatsapp_sync_jobs / _job_logs, dispatcher_heartbeat, view uazapi_history_pending_by_client. waba_templates, instagram_config.
- **Jurídico**: datajud_monitored_processes / _alerts / _notification_config. Advbox vive no banco externo.
- **IA/Geração de conteúdo**: generation_agent_prompts / _prompt_versions / _prompt_cases, generation_legal_cases / _versions, generation_templates / _template_versions, generation_prompt_config; ai_provider_keys, ai_usage_logs, client_ai_model_config / _list.

- **Billing:** `julia_orders`, `julia_plans`, `julia_payment_config`, `julia_contract_template`; `contract_notification_configs` / `_logs`, `contract_deletion_audit`.
- **Ajuda:** `help_posts` / `_categories` / `_post_views` / `help_studio_editors`.
- **Notificações:** `push_notifications`, `push_subscriptions`, `internal_notifications` / `_recipients`.
- **Misc:** `user_avatars`, `module_embeds`, `quick_messages`, `link_preview_cache`, `webhook_logs`, `webhook_queue`.
- **Funções RPC:** `clear_user_presence`, `get_infra_stats`, `generate_ticket_protocol`, `gen_wavoip_friendly_code`, `map_priority_chat_to_crm` / `_crm_to_chat`, `get_return_chat_candidates`, `get_task_ranking`, `get_team_online_seconds_by_day`, `increment_contact_unread`, `increment_help_post_view`.

Postgres Externo (legado — dados core de agentes/leads)

Acesso **exclusivo** via edge function `db-query` (`supabase/functions/db-query/index.ts`, ~3600 linhas) + wrapper `src/lib/externalDb.ts` (classe `ExternalDatabase`, singleton `externalDb`).

Conexão: driver `postgresjs`. Pool singleton por isolate (`max:5`, `prepare:false`, `connect_timeout:15s`, `statement_timeout:30s`, `timezone America/Sao_Paulo`). SSL via `EXTERNAL_DB_CA_CERT` ou `require`. Porta default 25061 (managed DB DigitalOcean). Retry com backoff no server e no cliente (`externalDb.invoke`, até 5 tentativas com jitter, detectando 502/503/504).

Tabelas (deduzidas das queries):

- `users` — `id`, `name`, `email`, `password` (bcrypt), `role`, `cod_agent`, `client_id`, `user_id` (FK auto-referente = usuário pai/titular), `evo_url/evo_instance/evo_apikey/hub`, `data_mask`, `use_custom_permissions`, `is_active`, `remember_token`.
- `clients` — `tenant real`: `name`, `business_name`, `federal_id` (CNPJ), `email`, `phone`, `endereço`, `photo` (logo).
- `agents` — agente IA de WhatsApp: `cod_agent`, `client_id`, `user_id` (owner), `settings` (jsonb), `prompt`, `is_closer`, `agent_plan_id`, `due_date`, `status`, `hub/evo_*/waba_id/waba_token/waba_number_id`, `last_used`.
- `user_agents` — N:N usuário↔agente: `user_id`, `agent_id` (NULL=monitora), `cod_agent`, `can_edit_prompt`, `can_edit_config`.
- `agents_plan` — `name`, `limit` (leads), `satus` [sic] (bool ativo).
- `sessions` — `id`, `whatsapp_number`, `agent_id`, `active`.
- `log_messages` — `session_id`, `created_at` (conta leads do mês).
- `crm_atendimento_cards` / `crm_atendimento_stages` — CRM de atendimento (kanban por telefone): `whatsapp_number`, `cod_agent`, `stage_id`, `card_id`.
- `campaing_ads` [sic] — leads de Meta Ads: `campaign_data` (jsonb), `session_id`, `created_at`.
- `modules` / `user_permissions` / `role_default_permissions` — sistema de permissões (criadas via `init_permission_system`).
- View `vw_equipe` — equipe por `client_id` (roles admin/user/colaborador/time/advogado/comercial).
- Tabelas Advbox: `advbox_integration`, `advbox_rules`, `advbox_processes_cache`, `advbox_notification_logs`, `advbox_client_queries`, `advbox_lead_syncs`.
- Views de performance (usadas em Dashboard/Estratégico): `vw_painelv2_desempenho_julia_all`, `vw_painelv2_desempenho_julia`.

Actions do db-query (~120, todas via `sql.unsafe(query, params)` parametrizado):

- CRUD genérico: `select` (colunas/orderBy **interpolados sem sanitização** — risco se o chamador passar valor não confiável), `insert` / `update` / `delete` (com RETURNING; `agents.settings` tem tratamento especial de JSONB).
- `raw` — **executa SQL arbitrário**: `sql.unsafe(query, params)` sem allowlist. Exposto no frontend via `externalDb.raw({query, params})`. **Superfície de risco crítica.**
- Auth/tenant: `login` (bcrypt, herda `client_id` do pai), `change_password`, `reset_user_password`, `get_effective_client_id` (COALESCE user→pai→user_agent), `get_client` / `update_client` / `insert_client` / `delete_client`, `search_clients`, `check_federal_id_exists`.
- Agentes/equipe: `get_user_agents`, `get_agents_list`, `search_agents`, `get_next_agent_code`, `insert_agent` / `update_agent` / `delete_agent` / `get_agent_details` / `get_agent_by_cod`, `insert_user_agent` / `delete_user_agent` / `update_user_agent_ownership` / `update_user_agent_permissions`, `update_agent_connection` / `update_agent_waba_connection`, `get_team_by_client` / `get_team_members`, `insert_team_member` / `update_team_member` / `delete_team_member`, `get_crm_agents_for_user`, `get_plans`.
- Sessões/CRM/chat: `get_session_status` / `update_session_status` / `get_inactive_sessions`, `chat_bootstrap` (agregador: campanhas Meta Ads + etapa CRM + status de sessão em 1 round-trip, com lógica de variantes de telefone BR), `followup_stop`.
- Permissões/módulos/filas: `init_permission_system` (cria as tabelas), `get_user_permissions` / `get_modules` / `get_menu_modules` / `create_module` / `update_module` / `delete_module`,

```
get_role_default_permissions / update_role_default_permissions , update_user_permissions / get_users_with_permissions ,
init_queue_access_system / get_user_queue_access / set_queue_members , init_embed_system / resolve_module_embed .
```

- Advbox: `advbox_*` (get/save/delete integration, rules, processes, notification_logs, client_queries, lead_syncs).
- Diagnóstico: `ping`, `get_external_infra_stats`, `diagnose_agents_settings` / `normalize_agents_settings`.

Tratamento de settings JSONB: `normalizeSettings` desfaz até 3 níveis de double-stringify; inserts/updates usam `CASE WHEN jsonb_typeof(...)='string' THEN (v #>> '{}')::jsonb ELSE v END`.

Autenticação & Autorização

Arquivos: `src/contexts/AuthContext.tsx`, `src/types/permissions.ts`, `src/lib/resolveEffectiveClientId.ts`.

Fluxo de login:

1. `externalDb.login(email, password)` → `bcrypt` no `db-query` (nunca no browser).
2. `is_active===false` → bloqueia.
3. `checkVersionAndReloadIfNeeded()` antes de montar estado.
4. `hydrateClientPhoto` → `setUser` → persiste `localStorage` (`AUTH_USER`, `AUTH_LAST_ACTIVITY`).
5. `loadPermissions(userId)` → `createPermissionMap`.
6. Telemetria não-bloqueante: `logUserActivity('login')` + `collectClientEnvironment` / `logUserDevice`.

Objeto User: `id`, `name`, `email`, `role`, `cod_agent?`, `client_id?`, `user_id?`, `evo_url/evo_instance/evo_apikey?`, `data_mask?`, `hub?`, `avatar?`, `client_name?`, `use_custom_permissions?`, `is_active?`.

Roles: `AppRole` = `admin` | `colaborador` | `user` | `time` | `advogado` | `comercial`. `admin` sempre passa em `hasPermission`. `admin` / `user` são titulares; `time` / `advogado` / `comercial` são membros de equipe.

Permissões por módulo: `ModuleCode` (~45 codes fixos + | `string` para embeds dinâmicos). `UserPermission` {`module_code`, `module_name`, `category`, `can_view`, `can_create`, `can_edit`, `can_delete`}. `hasPermission(moduleCode, action='view')`: `admin`→`true`; senão consulta o mapa. Custom por usuário (`use_custom_permissions`) ou herdado de `role_default_permissions`.

resolveEffectiveClientId: `user.client_id` se existir; senão `externalDb.getEffectiveClientId(id)`; senão varre `getUserAgents` procurando o 1º `client_id`. Mesma lógica no `restoreSession`.

Inatividade/logout: `INACTIVITY_TIMEOUT_MS` = 30min (const real — comentários no código dizem "1h", desatualizados). Rastreia `mousemove/mousedown/keydown/touchstart/scroll/wheel` + `visibilitychange`, throttle 5s. Checagem a cada 30s. Expira → `logUserActivity('logout_inactivity')`, `clearPresence` (RPC `clear_user_presence`), limpa `storage`, `window.location.replace('/login')`. Sincroniza entre abas via `storage event`.

Avatar: prioridade `user_avatars.photo_url` (Supabase, por usuário) → fallback `clients.photo` (externo). Dois caches `localStorage`.

Multi-tenancy

Isolamento é feito na APLICAÇÃO, não no banco.

- Chave de tenant = `client_id` (tabela `clients`, externa). Sub-usuários herdam via `COALESCE` no `login / get_effective_client_id / resolveEffectiveClientId`.
- Filtragem ocorre nas queries do `db-query` e nos hooks do frontend.
- **Supabase RLS é permissiva:** `CREATE POLICY ... FOR ALL USING (true) WITH CHECK (true)` — confirmado em ~100 migrations (269 ocorrências de `USING(...)` em 100 arquivos, padrão dominante `true`). RLS habilitada mas não restringe nada — isolamento depende só da app.
- **Consequência:** `db-query` com `verify_jwt` default + `raw / select` genéricos + RLS `true` = modelo de tenant "confiança no cliente".

Edge Functions (116 diretórios em `supabase/functions/`)

Ver `supabase/config.toml` para o mapa completo de `verify_jwt`. Resumo por domínio (ver docs específicos para detalhe funcional):

- **Core:** `db-query` (gateway externo).
- **Chat/Inbox:** `chat-webhook-dispatcher`, `chat-public-api / webchat-api`, `chat-ai-assist / chat-ai-process`, `chat-automation-engine`, `chat-campaign-dispatcher`, `chat-route-conversation`, `chat-scheduler`, `chat-return-chat`, `chat-bulk-`

- `close`, `chat-contacts-enrich`, `chat-media-upload / download`, `chat-transcribe-audio`, `chat-message-react`, `chat-reset`, `team-member-cleanup-conversations`, `link-preview`, `image-proxy`.
- **uazapi/WhatsApp**: `uazapi-proxy` [no-jwt], `uazapi-chat-webhook`, `uazapi-admin`, `uazapi-instance-manager`, `uazapi-chat-backfill`, família `uazapi-history-*` [dispatcher/heartbeat no-jwt].
- **WABA**: `waba-admin / waba-send` [no-jwt], `waba-templates`, `meta-webhook` [no-jwt], `meta-auth`, `meta-send-test`.
- **Instagram**: `instagram-webhook`, `instagram-send`.
- **Meta Ads**: `meta-ads` [no-jwt], `meta-conversions` [no-jwt].
- **CRM/Copilot IA**: `copilot-chat`, `crm-copilot-monitor`, `prompt-generator`, `batch-generate-scripts`.
- **Advbox**: `advbox-integration`, `advbox-sync`, `advbox-notify` [no-jwt], `advbox-query` [no-jwt].
- **DataJud/documentos**: `datajud-search / datajud-monitor` [no-jwt], `consulta-documento`, `zapsign-file / zapsign-download`.
- **Telefonia**: `api4com-proxy / webhook` [no-jwt], `threecplus-proxy / webhook` [no-jwt], `vellip-webhook` [no-jwt], `telephony-order-create / checkout`, `telephony-provision`, `telephony-notify-paid`.
- **Wavoip**: `wavoip-provision-device / connect-device` [no-jwt], `wavoip-device-provision`, `wavoip-disconnect/rename-device`, `wavoip-call-webhook` [no-jwt], `wavoip-configure/verify-webhook`, `wavoip-fetch-recording` [no-jwt], `wavoip-fetch-call-details`, `wavoip-transcribe-recording`, `wavoip-sync-history`, `wavoip-reconcile-call / runner`, `wavoip-providers`.
- **Filas**: `queue-order-create / checkout` [no-jwt], `queue-provision` [no-jwt], `queue-management`, `queue-maintenance`, `queue-resolve-phone`, `sync-queue-to-agent`.
- **Vídeo**: `video-order-create / checkout`, `video-provision`, `video-room`.
- **Pagamentos**: `asaas-checkout / webhook` / `configure-webhook` [no-jwt], `infinitypay-checkout / webhook` [no-jwt], `mercadopago-checkout / webhook`.
- **Suporte/tickets**: `support-assistant-webhook` [no-jwt], `support-transcribe-audio` [no-jwt], `support-group-discovery` [no-jwt], `ticket-media-upload`.
- **Notificações**: `send-push` [no-jwt], `internal-notification-dispatch / scheduler` [no-jwt], `contract-notifications-cron` [no-jwt] / `queue`.
- **IA/config**: `ai-provider-key-set` [no-jwt], `client-automation-flags`.
- **Cron/infra**: `assigned-user-id-backfill / cron` [no-jwt], `telemetry` [no-jwt], `n8n_execute / n8n_execute-followup-stop` [no-jwt].

Versionamento

`vite-plugin-auto-version.ts` (autoBumpVersion, só no build): MAJOR.MINOR = maior entre `package.json` e `public/version.json`; PATCH auto-incrementa a cada build. `versionFilePlugin` grava `dist/version.json` no `closeBundle`. `src/lib/appVersion.ts` (`checkVersionAndReloadIfNeeded`): compara `/version.json` com `__APP_VERSION__`; se diferente → toast + `forceReloadForNewVersion()` (desregistra SW, limpa caches, preserva só `AUTH_USER` / `AUTH_LAST_ACTIVITY`, recarrega com `?v=timestamp`). Pula em preview hosts. Chamado no `login()` antes de montar o app.

Roteamento (`src/App.tsx`)

Providers aninhados: `QueryClientProvider` → `TooltipProvider` → `DebugProvider` → `BrowserRouter` → `AuthProvider` → `UaZapiProvider` → `WavoipProvider` → `ErrorBoundary` → `Suspense`. Rotas lazy exceto `Login / Dashboard / NotFound / RedirectPage / ComprarPage / JoinCallPage`. `MainLayout` envolve o grupo autenticado.

Rotas públicas: `/login`, `/redirect`, `/comprar`, `/comprar/sucesso`, `/call/:roomName`, `/tv/master` (só auth).

Rotas autenticadas (dentro de `MainLayout`) e seu `ProtectedRoute` `module=`:

Rota	Módulo
/ , /dashboard	— (só auth)
/crm/leads	crm_leads
/crm/lead-estatisticas	crm_statistics
/crm/lead-monitoramento	crm_monitoring
/estrategico/desempenho , /estrategico/campanhas	strategic_perf
/estrategico/contratos	strategic_contract
/agente/meus-agentes(/:codAgent/editar)	agent_management
/agente/filas	filas
/agente/followup	followup
/configuracoes	configuracoes
/chat + 24 sub-rotas	sem proteção de módulo
/biblioteca	library
/equipe	team
/advbox/*	sem proteção de módulo
/crm-builder(/:boardId)	crm_leads
/datajud	datajud
/casos-juridicos	legal_cases
/notificacoes-contrato	contract_notifications
/telefonias	telephony
/comercial/crm	crm_comercial
/suporte-assistente	support_assistant
/mensagens-rapidas	quick_messages
/atendimento-humano	human_support
/contatos	contacts
/tarefas	tasks
/notificar-clientes	notify_customers
/tickets(/:id)	support_tickets
/ajuda/*	help_center
/admin/agentes* , /admin/modulos , /admin/permissoes , /admin/meta-* , /admin/monitoramento , /admin/operacoes , /admin/webhook-monitor , /admin/contrato- template , /admin/video	admin_agents
/admin/copiloto	copilot_admin
/admin/telefonias	telephony_admin
/wavoip	wavoip
/admin/wavoip	wavoip_admin
/admin/chat	chat_admin
/admin/prompts	prompt_generator
/admin/pedidos	julia_orders
/admin/planos	julia_plans

Rota	Módulo
/admin/notificacoes-push	push_notifications
/admin/embeds	admin_embeds
/sys/:code	— (embeds dinâmicos via resolve_module_embed)
/telefonica/contratar , /filas/contratar , /video/contratar	— (só auth)

Achados-chave

- 1. Arquitetura híbrida: Supabase (~177 tabelas) + Postgres externo legado, acessado só via db-query .
- 2. Auth própria (bcrypt), não Supabase Auth; sessão localStorage ; timeout 30min.
- 3. Multi-tenancy por aplicação; RLS Supabase permissiva não isola tenant no banco.
- 4. Superfície de risco: raw (SQL arbitrário) + select genérico sem sanitização + 40+ functions sem verify_jwt .
- 5. Permissões: módulo × ação, defaults por role ou custom por usuário, tabelas criadas dinamicamente.
- 6. Versionamento auto-bump de PATCH + force-reload no login.
- 7. Chat é o maior domínio (~50 tabelas, 24 sub-rotas) mas sem proteção de módulo no router.

Módulo Chat / WhatsApp

Maior domínio funcional do sistema. Inbox omnichannel com atribuição, filas, SLA, IA e integrações de telefonia.

Detalhe interno do webhook uazapi (parsing, dedup, pipeline de histórico assíncrono): <docs/uazapi-integration.md>.

1. Provedores / Canais

Campo `channel / channel_type`: `whatsapp_uazapi` (WhatsApp não-oficial via uazapi), `whatsapp_waba` (WhatsApp Cloud API oficial Meta), `webchat` (widget próprio), `instagram` (Direct via Meta Graph). Badge em `src/components/chat/ChatHeader.tsx` (`ChannelBadge`).

- **uazapi**: `uazapi-chat-webhook` (recebimento), `uazapi-proxy` (envio genérico), `uazapi-instance-manager`, `uazapi-admin`, `uazapi-chat-backfill`, família `uazapi-history-*` (importação assíncrona).
- **WABA (oficial)**: `waba-send` (envio), `waba-admin`, `waba-templates`, `meta-webhook` (compartilhado com Instagram/Meta Ads).
- **Instagram**: `instagram-webhook`, `instagram-send`.
- **WebChat**: `webchat-api`, `chat-public-api`.

Envio (frontend → função → provedor) — `src/contexts/WhatsAppDataContext.tsx`

- Texto uazapi: `uazapi-proxy` `endpoint: '/send/text'`, `token=queue.evo_apikey`, `baseUrl=queue.evo_url`, `body {number, text, replyid, forward}`. Resposta: `data.key.id` (stanza WhatsApp → `message_id`), `data.id/messageId` (id interno uazapi → `external_id`).
- Texto WABA: `waba-send` `action: 'send_text'`, `queue_id`, `to`, `text`.
- Mídia uazapi: `/send/media`; áudio manda base64 direto; demais tipos mandam URL já persistida (`chat-media-upload`).
- Mídia WABA: `waba-send` `action: 'send_media'` com `mediaBase64`. Áudio WebM é reconvertido para OGG/Opus estrito (WABA exige contêiner OGG real).
- **Grupos**: uazapi aceita `@g.us`; **WABA rejeita explicitamente** (erro lançado no `sendMessage / sendMedia`).
- Edição: só uazapi (`/message/edit`).
- Mark-as-read: uazapi `/chat/markRead`; WABA `action: 'mark_read'` via `message_id` (`wamid`).
- Reação: `chat-message-react` grava em `chat_message_reactions`.

2. Modelo de Dados (Supabase)

Schema evoluiu por migrations incrementais (`ALTER TABLE ADD COLUMN IF NOT EXISTS`) — colunas usadas no frontend podem não estar no `CREATE TABLE` original.

`chat_contacts`

`id`, `client_id`, `cod_agent`, `phone`, `name`, `avatar`, `is_group`, `is_archived`, `is_muted`, `unread_count`, `last_message_at`, `last_message_text`. Adicionadas depois: `channel_source` (UUID da fila — chave de roteamento), `channel_type`, `remote_jid`. Upsert on conflict `phone, client_id`.

`chat_messages`

`id`, `contact_id` (FK CASCADE), `client_id`, `message_id` (id externo/wamid, indexado), `text`, `type` ('text' default), `from_me`, `status` ('sent' default), `media_url`, `file_name`, `caption`, `reply_to`, `metadata` (jsonb), `timestamp`, `edited_at`. Adicionadas depois: `external_id` (id interno do provedor, dedup em paralelo a `message_id`), `conversation_id` (FK), `internal_note`, `note_type`, `sender_name`, `is_forwarded`, `forwarded_score`, `channel_type`. Tipos (`type`): text, image, video, audio, ptt, document, location, contact, sticker, revoked.

`chat_conversations`

`id`, `contact_id` (FK), `client_id`, `cod_agent`, `channel` ('whatsapp_uazapi' default), `status` ('pending' default: pending|open|resolved|closed), `protocol` (auto via trigger `#YYYY-000001`), `assigned_to`, `assigned_user_id`, `department`, `priority` ('normal': urgent|high|normal|low), `tags[]`, `opened_at`, `first_response_at`, `closed_at`, `resolved_at`, `close_reason`, `close_note`, `metadata`. Adicionadas depois: `queue_id` (FK filas), `snoozed_until`, `snooze_reason`, `snoozed_by`, `active_ticket_id`, `active_ticket_number`, `active_ticket_protocol`. Trigger `trg_generate_conversation_protocol`.

chat_conversation_history

id, conversation_id (FK CASCADE), action (opened|reopened|closed|resolved|assigned|tag_added|tag_removed|note_added|snoozed|returned_to_queue), actor_name, from_value, to_value, notes, created_at + user_id, to_user_id, from_user_id. Log de auditoria/timeline, intercalado com mensagens em ChatMessages.tsx via ConversationEvent.

chat_tags / chat_conversation_tags

chat_tags(id, name, color, client_id); junção chat_conversation_tags(conversation_id, tag_id).

queues

id, client_id, name, channel_type ('uazapi' default: uazapi|waba), hub, evo_url/evo_apikey/evo_instance (uazapi), waba_id/waba_token/waba_number_id, is_active, is_deleted, deleted_at. Adicionadas depois: phone_number, phone_resolved_at, waba_webhook_status/last_error/subscribed_at, settings (jsonb).

queue_agent_links

id, queue_id (FK RESTRICT), cod_agent, is_primary. **Constraints:** UNIQUE(queue_id, cod_agent); UNIQUE(cod_agent) WHERE is_primary=true (um agente só pode ter UMA fila primária no sistema todo). Sincronizado para o agente externo via sync-queue-to-agent (propaga credenciais via update_agent_connection / update_agent_waba_connection).

Outras tabelas de suporte

- chat_message_reactions: message_id, external_message_id, reactor, emoji, from_me.
- chat_csat_responses / chat_csat_config: pesquisa pós-encerramento (score 0-5, status: sent|answered).
- chat_scheduled_messages: client_id, cod_agent, contact_id, conversation_id, text, scheduled_for, created_by(_name) — processada por chat-scheduler.
- chat_conversation_presence: presença persistida por conversa.
- chat_sla_configs: client_id, cod_agent, priority, first_response_minutes, nrt_response_minutes, resolution_minutes, is_active — unique (client_id, priority).
- chat_campaigns / _recipients / _variants (A/B) / _schedules.
- user_presence / _heartbeats (particionada por data) / _daily.
- chat_routing_rules / chat_agent_capacity.
- chat_client_settings: JSONB (auto_transcribe_audio, events_enabled, event_visibility).
- chat_conversation_summaries: resumos IA (sentiment, summary, atendimento, message_count, triggered_by).
- uazapi_history_runs / _items, whatsapp_sync_jobs / _job_logs: jobs assíncronos.
- wavoip_device_queues: dispositivos Wavoip ↔ filas.

3. Fluxo de Conversa**Recebimento (uazapi) — uazapi-chat-webhook**

Dedup em duas camadas: (1) match exato message_id OU external_id (índice btree); (2) fallback ilike por sufixo (namespaces diferentes entre provedores). Pré-filtro em lote antes de inserir (evita duplicar em batch de histórico). Persiste chat_messages, upsert chat_contacts, resolve/reabre chat_conversations (reabre resolved recente mantendo assigned_to ou volta a pending). Edição de mensagem recebida faz UPDATE casando message_id.eq OR external_id.eq. Descarte não roteável logado via logDroppedMessage (_shared/droppedLogger.ts).

Recebimento (Realtime frontend) — WhatsAppDataContext.tsx

3 canais Supabase Realtime: chat_contacts, chat_messages (dedup via knownMessageIds ref), chat_conversations. No INSERT inbound (!from_me && !internal_note): incrementa unread_count via RPC atômico increment_contact_unread (evita race entre agentes simultâneos), reposiciona contato, dispara chat-automation-engine (message_received) e chat-webhook-dispatcher (message_received) fire-and-forget.

Envio

Mensagem otimista (`status: 'sending'`) local antes da chamada; sucesso → `sent` com `message_id / external_id`; erro → `failed` (toast via `normalizeSendError()` : timelock uazapi, janela 24h WABA expirada, número inválido, rejeição genérica).

Atribuição / Transferência / Devolução

- **Assumir** (`handleTakeOver`): `assignConversation(id, userName, userId)` seta `assigned_to / assigned_user_id`; `pending=open`; evento `assigned` em histórico; desativa a Julia (`disableJuliaOnAssignOrTransfer` — desliga sessão via `externalDb.getSessionStatus + n8n_execute-followup-stop`).
- Primeira resposta também auto-atribui se ainda não houver `assigned_to`, seta `first_response_at + status='open'`.
- **Transferir** (`TransferDialog.tsx`): seleciona membro do time, `assignConversation`.
- **Devolver à fila** (`ReturnToQueueDialog.tsx`): zera `assigned_to / assigned_user_id`, força `pending`, evento `returned_to_queue`.
- **Claim guard** (`ChatInput.tsx`): só permite enviar se `noteMode` OU (`isAssignedToMe && status in [pending, open]`); banner "Assuma esta conversa" quando não atribuído.

Snooze — `SnoozeDialog.tsx`

Presets (1h, 4h, amanhã 9h, 2 dias, 1 semana) ou custom; `snoozed_until/reason/by` + evento `snoozed`. Aba "todas" esconde contatos com `snoozed_until` futuro.

Resolver / Encerrar / Reabrir

`updateConversationStatus()`: `resolved` → `resolved_at`; `closed` → `closed_at (+ close_note)`; reabertura limpa `closed_at / resolved_at`. Em `resolved/closed` dispara `chat-webhook-dispatcher (conversation_resolved)` e `chat-ai-assist (incremental_summary, nota interna automática)`. **CSAT ao encerrar** (`CSATDialog.tsx`): nota de encerramento opcional; se "enviar pesquisa", insere placeholder em `chat_csat_responses` + envia mensagem de pesquisa.

4. Filas (Queues)

- `queues` guarda credenciais por canal; toda `chat_conversations` tem `queue_id`.
- `queue_agent_links` vincula fila↔agente-IA (`cod_agent`), máx. 1 fila primária/agente. Hooks `useQueueAgentLink(s) / useAgentQueueLink`.
- **Sincronização** `sync-queue-to-agent`: credenciais da fila → registro do agente externo (`update_agent_connection / update_agent_waba_connection`). Webchat/Instagram não sincronizam.
- **Resolução de fila efetiva por contato** (`getEffectiveQueue`): 1) conversa ativa em memória, 2) conversa mais recente com `queue_id`, 3) `contact.channel_source` (se UUID), 4) qualquer fila ativa do canal, 5) fila selecionada no topo.
- **Roteamento automático** (`useChatRouting.ts`): `chat_routing_rules` (condições `channel/tag/priority/keyword/business_hours/queue/contact_is_new`; estratégias `round_robin/least_busy/specific_agent/manual_pool/random`) + `chat_agent_capacity` (status online/away/busy/offline, `max_concurrent, current_load`). Executado por `chat-route-conversation + chat-automation-engine`.
- **Acesso por fila**: `useUserQueueAccess / queue_members` restringe filas visíveis por atendente.

5. Recursos

- **Notas internas**: `sendInternalNote()` → `internal_note:true, note_type` (info/question/urgent). Suportam @menções (`src/lib/chat/mentions.ts`). Escopadas por `conversation_id`.
- **Citadas/quoted**: `metadata.quoted_message{id,text,from_me,sender_name,type}`; fallback via `withDerivedQuote()` indexando `message_id/external_id/id`.
- **Encaminhadas**: `ForwardDialog.tsx`, flag `forward:true`, marca `is_forwarded:true`.
- **Agendadas**: `ScheduleMessageDialog.tsx` → `chat_scheduled_messages`; processadas por `chat-scheduler` (cron).
- **Grupos**: `chat_contacts.is_group`; abas individual/grupos (persistido `localStorage chat:activeTab`); WABA bloqueia envio a grupos.
- **Busca**: `ChatSearchDialog.tsx` (na conversa) + busca global por nome/telefone (variantes BR via `src/lib/phoneVariants.ts`).
- **Mídia**: `chat-media-upload` (outbound antes de enviar), `chat-media-download` (inbound criptografada; retry state machine `idle|loading|success|transient_failed|permanent_failed`, backoff 30s, máx 3). `MediaLightbox.tsx`.
- **Reações**: `chat-message-react` → `chat_message_reactions`; Realtime via `useMessageReactions.ts`.
- **Campanhas em massa**: `chat-campaign-dispatcher` (variantes A/B, agendamento), `chat-bulk-close`.

- **Transcrição de áudio:** `chat-transcribe-audio` — manual ou automática (gate duplo: flag do cliente E da fila via `chat_client_settings / useQueueAutomationFlags`).
- **IA:** `chat-ai-assist` (incremental_summary no fechamento, full_summary sob demanda → `chat_conversation_summaries`), `chat-ai-process`, `chat-return-chat` (worker "Retornar Chat"). `copilot-chat` é módulo separado.
- **SLA:** `useChatSlaConfigs.ts`, defaults hardcoded por prioridade. `evaluateSla()`: 3 fases — **FRT** (aguardando 1ª resposta desde `opened_at`), **NRT** (última msg do cliente, aguardando resposta desde `last_customer_message_at`), **TTR** (aguardando resolução desde `opened_at`); status `on_track|at_risk (≤25% restante)|breached`. `SlaBadge.tsx`.
- **Presença:** `useConversationPresence.ts` (Realtime Presence efêmero, sem DB) + `useGlobalPresence.ts` (singleton por `client_id`). Separado: `user_presence_heartbeats` (telemetria histórica).
- **Dropped messages:** `_shared/droppedLogger.ts` — auditoria de mensagens não persistidas (roteamento falho etc).
- **Histórico uazapi:** pipeline assíncrono `uazapi-history-dispatcher / -processor / -warmup / -resume / -cancel / -force-resync / -dispatcher-heartbeat` sobre `uazapi_history_runs / _items`.
- **Telefonia integrada:** `WaveipCallButton.tsx / WaveipCallDialog.tsx` + família `waveip-*`; alternativa SIP nativa (`PhoneContext / usePhone`); `UpsellCallDialog.tsx` quando VOIP indisponível.

6. Vínculo Chat ↔ Ticket

- `chat_conversations.active_ticket_id/_number/_protocol` mantidos por trigger de banco em `support_tickets`.
- `useTicketLinkedConversations.ts`: mapeia `conversation_id` → `TicketConversationLink`; invalida via Realtime em mudanças de `support_tickets`.
- Botão "Abrir ticket" em `ChatHeader.tsx`: se já existe ticket vinculado + permissão de ver → `ChatTicketDetailSidePanel.tsx`; senão, com permissão de criar → `ChatTicketSidePanel.tsx`.
- `ChatTicketSidePanel.tsx` (painel lateral, portal): form pré-preenchido (nome/telefone bloqueados), responsável, departamento/categoria, prioridade. Ao criar: `conversation_id`, `contact_id`, `metadata:{is_group, group_jid, source:'chat'}`.

Integração UaZapi — Detalhe Técnico

Complementa [docs/chat.md](#). Cobre o parsing interno do webhook e o pipeline de histórico assíncrono.

uazapi-chat-webhook/index.ts (~1800 linhas) — eventos de entrada

POST ?queue_id={id}. Resolve `queues (id, client_id, channel_type, evo_url, evo_apikey, waba_token, waba_number_id)`.

Aliasing de evento: `EVENT_ALIAS` mapeia nomes uazapi (underscore) → nomes Evolution/Baileys (dot): `messages_update→messages.update`, `connection→connection.update`, etc.

Branches

- **connection.update:** só log.
- **messages.update** (status + edições): `collectMessageIds` junta ids de vários campos possíveis; `resolveChatMessageRowIds` faz match exato + fallback `ILIKE` por sufixo (ids prefixados tipo `3EB0:xxx`). Edição atualiza `text` + `edited_at`. Status via `STATUS_MAP` (`'0'→failed`, `'1'→pending`, `'2'→sent`, `'3'→delivered`, `'4'/'5'→read`) com `STATUS_RANK` garantindo que status **nunca regride** (ex.: `read→delivered` é bloqueado).
- **messages.delete:** `type='revoked'`, `text='🚫 Mensagem apagada'`.
- **contacts.update / groups.update / chats.update:** atualiza nome/avatar/arquivado/mutado/não-lidas; dispara `refresh-contact-avatar` em background quando a foto muda.
- **history/messages.set:** extrai mensagens, enfileira via `enqueueHistoryRun` (não processa inline).

Loop principal (mensagem por mensagem)

1. **chatId:** de vários campos possíveis. **Grupo** = presença de `@g.us` na JID (estrito, ignora flags `isGroup` para evitar falso positivo).
2. **Gate de grupo:** se grupo e `chat_client_settings.settings.ALLOW_GROUPS=false` (cache 60s) → descarta + `logDroppedMessage(reason='group_blocked')`.
3. **messageId** ausente → descarta (`reason='no_id'`).
4. **Edição via upsert:** detecta `protocolMessage.editedMessage`; se identificado, UPDATE em vez de INSERT.
5. **Dedup:** lookup `client_id + (message_id.eq OR external_id.eq)`; se existe, aplica bump de status (nunca regride) e `continue`.
6. **Resolução de telefone do par:** para grupos usa o id do grupo; para individual escaneia candidatos (`fromMe` usa `to / recipient`, senão `sender_pn/PhoneNumber/phone/from/sender/...`), rejeita `@lid / @g.us`, normaliza 8-13 dígitos (regra BR do 9º dígito). Sem telefone → descarta.
7. **Upsert de contato** (`chat_contacts`, `conflict phone,client_id`): `channel_type='whatsapp_uazapi'`, `channel_source=queueId`, `remote_jid` normalizado, `unread_count` incrementado só se `!fromMe`. Lógica evita sobrescrever nome real com nome tipo-telefone.
8. **Enriquecimento de perfil:** fire-and-forget `fetchWhatsappProfile` atualiza avatar/nome.
9. **Backfill 1x:** contato novo não-grupo → dispara `uazapi-chat-backfill` (limit 50).
10. **Resolução de conversa:** (1) ativa `pending / open`; (2) reabre `resolved` mais recente; (3) cria nova `pending`. Race fallback re-seleciona.
11. **Reações: não** vão para `chat_messages` — resolvem alvo por `message_id`, deletam reação anterior do mesmo `reactor`, inserem em `chat_message_reactions`.
12. **Quoted/reply:** `resolveQuotedMeta` (`_shared/quotedMessage.ts`) monta `{id,text,from_me,sender_name,type}` do original ou de conteúdo embutido.
13. **Insert de mensagem:** `chat_messages` com `metadata` incluindo `sender_id`, `sender_name`, `is_ptt`, `duration`, `mimetype`, `quoted_message`. Duplicado (23505) tolerado.
14. **Auto-transcrição de áudio:** se `AUTO_TRANSCRIBE_AUDIO` ligado (client + fila), dispara `chat-transcribe-audio`.

Extração de mídia: `extractMessageType` mapeia `mediaType/mimetype` para `text/image/video/audio/ptt/document/sticker/location/contact/reaction/revoked` (GIF→video). `extractMediaUrl` prefere `content.URL`, depois `fileURL/mediaUrl` decifrado, evita `.enc`. **O webhook não baixa mídia** — só grava `media_url`; download real é via `chat-media-download`, só disparado no caminho de importação de histórico.

Auditoria de descarte: `_shared/droppedLogger.ts` (`logDroppedMessage`) grava em `chat_dropped_messages` (`reason: no_phone|group_blocked|no_id|group_no_id`). Fire-and-forget, nunca quebra o webhook.

uazapi-proxy/index.ts — passthrough genérico de envio

POST {method, endpoint, body, token, baseUrl}. Whitelist de prefixos: /chat/, /message/, /send/, /instance/, /labels/, /group/, /call/, /business/, /chatwoot/, /community/. Sem acesso a banco — é onde o frontend manda mensagens de saída.

uazapi-instance-manager/index.ts — lifecycle por fila

Actions: create (/instance/init + configura webhook), reconfigure_webhook(_all), connect (QR), status, disconnect, delete. DEFAULT_WEBHOOK_EVENTS = connection, messages, messages_update, history, chats, contacts, groups, presence, call.

uazapi-admin/index.ts — gestão a nível de agente (banco externo)

Conecta no Postgres externo direto. create_instance grava credenciais em agents (hub='uazapi', evo_url, evo_apikey, evo_instance). delete_instance, list_instances, create_instance_support (webhook para support-assistant-webhook, eventos ['messages']).

uazapi-chat-backfill/index.ts — histórico por contato sob demanda

POST {queue_id, contact_id, chat_id|phone, limit=50}. Pula se já backfilled ou grupo bloqueado. Chama {evo_url}/message/find. Upsert em chat_messages (conflict message_id, status='read', metadata.backfilled:true). Sempre marca history_backfilled=true mesmo em erro (evita retry loop).

Pipeline de Histórico (dump em tempo real)**Máquina de estados**

- **uazapi_history_runs**: 1 por dump (status pending|running|done|partial, contadores de mensagens/duplicatas/contatos).
- **uazapi_history_items**: 1 por chat dentro de um run (status: pending|running|ok|skipped|error, payload jsonb, attempts, worker_id, locked_at).
- **dispatcher_heartbeat**: saúde do pool de workers.
- Legado (importação em job): whatsapp_sync_jobs / _job_logs.

Fluxo

webhook.enqueueHistoryRun → insere run+items → uazapi-history-dispatcher (via cron heartbeat) mantém pool de até **10 workers** (uazapi-history-resume) → cada worker drena via RPC uazapi_pick_pending_items (SELECT FOR UPDATE SKIP LOCKED) → processa (dedup por external_id, status='read', unread_count sempre forçado a 0, batch de 100) → finalizeRunsCoalesced agrega.

- **uazapi-history-processor**: variante síncrona por payload direto ({run_id, payload}), agrupa por telefone real (rejeita @lid), skip incremental de mensagens mais antigas que last_message_at.
- **uazapi-history-dispatcher**: orquestrador persistente, ajusta pool (2/<100 pendentes, 5/<500, 10/senão), auto-reagenda tick em 10s.
- **uazapi-history-dispatcher-heartbeat**: cron de 1min, chama {action:'tick'} no dispatcher — rede de segurança se o isolate dele morreu.
- **uazapi-history-import**: importação em lote por job (whatsapp_sync_jobs), usa message_id prefixado backfill: {contactId}:{id} para dedup, dispara chat-media-download para mídia.
- **uazapi-history-warmup**: pede ao provedor (message/history-sync) para reenviar histórico via webhook.
- **uazapi-history-cancel**: seta cancel_requested=true em job rodando.
- **uazapi-history-force-resync**: desconecta/reconecta a instância para forçar o WhatsApp a re-empurrar histórico.

seed-uazapi-provider/index.ts

Upsert em queue_providers (provider_type='uazapi', credenciais de env). Bootstrapping inicial de um client novo.

Resumo de campos-chave

- **chat_messages**: sem `media_path` gravado pelo webhook/backfill/history — só `media_url` até `chat-media-download` rodar.
- **status**: `received` (inbound); `sent→delivered→read` (nunca regride); histórico/backfill sempre `read`.
- **direção**: bool `from_me`, sem coluna separada.
- **Endpoints uazapi usados**: `/instance/init|connect|status|logout|disconnect`, `/instance` (DELETE), `/instance/all`, `/admin/instances`, `/webhook`, `/message/find`, `/message/history-sync`.

CRM, Agentes de IA, Followup, Contratos, Jurídico

Dois clients de dados coexistem: `externalDb` (Postgres externo, via edge function `db-query`) para CRM de leads/agentes/followup/contratos; `supabase` nativo para CRM Builder/billing/config de IA.

1. CRM (Leads) — `src/pages/crm/`

Painel de leads captados via WhatsApp pelos agentes Julia, com pipeline de estágios. Fonte: Postgres externo via `externalDb.raw` (`src/pages/crm/hooks/useCRMData.ts`).

- `crm_atendimento_stages` — estágios do funil (`id`, `name`, `color`, `position`, `is_active`).
- `crm_atendimento_cards` — leads/cards (`id`, `helen_count_id`, `cod_agent`, `contact_name`, `whatsapp_number`, `business_name`, `stage_id`, `notes`, `stage_entered_at`, `owner_name`).
- `crm_atendimento_history` — histórico de mudança de estágio (`card_id`, `from_stage_id`, `to_stage_id`, `changed_by`, `changed_at`). Detecta `has_contract_history` (transição para estágios "Contrato em Curso"/"Contrato Assinado").
- `vw_painelv2_desempenho_julia_all / _julia` — views agregadas de conversas/sessões Julia, filtradas por `cod_agent` e período (America/Sao_Paulo).
- `externalDb.getCrmAgentsForUser(user.id)` → agentes visíveis (`cod_agent`, `owner_name`, `owner_business_name`).

Contrato vinculado a um lead (`ContractInfo`): `zapsing_doctoken` (token ZapSign), `status_document`, `signer_*`, `data_contrato`, `data_assinatura`, `resumo_do_caso`, `case_title/category`. `ContractInfoDialog.tsx`.

Followup integrado: `CRMFollowupInfo` (`step_number`, `node_count`, `followup_from/to`, `is_infinite`, `stage_label` ex. "1/4"/"∞/∞") — badge no card via `useFollowupActiveLeads.ts`.

Monitoramento (`CRMMonitoringPage.tsx`): `StuckLeadsAlert` (leads parados, `days_stuck`), `StageBottlenecks` (gargalos, `count` vs `avg_count`), `AgentWorkloadChart`, `ActivityTimeline`.

Estatísticas (`CRMStatisticsPage.tsx`): `ConversionFunnelChart`, `AverageTimeChart`, `AgentPerformanceTable` (`total_leads`, `qualified_leads`, `qualified_rate`, `contract_leads`, `contract_rate`, `avg_time_days`).

2. CRM Builder (Kanban) — `src/pages/crm-builder/`

Kanban customizável multi-board, **Supabase nativo** (separado do CRM de leads acima).

- `crm_boards`: `id`, `client_id`, `cod_agent`, `name`, `description`, `icon`, `color`, `position`, `is_archived`.
- `crm_pipelines`: colunas do board (`pipeline_id` referenciado por deals).
- `crm_deals` (select explícito de colunas — evita tráfego desnecessário em boards com milhares de deals): `id`, `board_id`, `pipeline_id`, `position`, `title`, `description`, `value`, `currency`, `priority`, `status`, `contact_name`, `contact_phone`, `contact_email`, `assigned_to`, `assigned_user_id`, `tags`, `due_date`, `expected_close_date`, `stage_entered_at`, `created_by`, `updated_by`, `cod_agent`, `client_id`, `custom_fields` (jsonb — inclui `links.chat/links.julia`).
- `status`: `'open'|'won'|'lost'|'archived'`. Listagem sempre `.neq('status','archived')`.
- `DealHistoryAction`: `'created'|'moved'|'updated'|'note_added'|'won'|'lost'|'archived'`.
- Posição ao criar: `max(position do pipeline) + 1`.
- Mover deal entre boards (`useMoveDealToBoard.ts`) marca o deal antigo `status:'archived'`.
- Realtime com `guard isMovingRef` + `debounce` para não sobrescrever estado otimista durante drag concorrente.
- Auditoria via `logCRMAudit()` (`useCRMAuditLog.ts`).

Painel de chat no card: `BoardChatSidePanel.tsx`, `DealJuliaPanel.tsx`, `useDealConversation.ts`, `useDealJuliaContext.ts` — vinculam deal a uma conversa/contato.

Outros: `CustomFieldsManager` (campos dinâmicos por board), `AutomationsManager` (`useCRMAutomations`), `BoardAnalyticsDashboard` (`PipelineFunnelChart`, `PipelineAvgTimeChart`, `DealsValueDistribution`).

3. Agentes de IA — `src/pages/agents/` + `src/pages/agente/`

Conceito central `cod_agent`: agente de IA configurado para um `client`, vinculado a um `user`, com plano, prompt, horário de atendimento e opcionalmente fila/instância WhatsApp.

Criação (`CreateAgentWizard.tsx`, `useAgentSave.ts`) — transacional com rollback manual:

- Valida CPF/CNPJ (`checkFederalIdExists`).

2. Cria/reaproveita `client` + `user` (senha temporária, `bcriptjs`).
3. Cria `agent`.
4. Em erro: `rollback()` deleta `agent/user/client` desta transação (só deleta `user/client` se não têm outros agentes vinculados).
5. `insertAgentChangeLog`; `ensureChatClientSettings`.

Wizard: `ClientStep`, `UserStep`, `PlanStep`, `ConfigStep`, `PromptStep`, `BusinessHoursEditor`.

Instâncias WhatsApp — `src/pages/agente/meus-agentes/`: `ConfigureInstanceDialog` + `useConfigureInstance` (provider `uazapi/WABA` via `ProviderSelector`), `QRCodeDialog` + `useQRCodePolling`, `ConnectionStatusBadge`, `WabaSetupDialog`, `useDeleteInstance`. Edge functions: `uazapi-instance-manager`, `uazapi-proxy`, `uazapi-admin`, `uazapi-chat-webhook`, `waba-admin`, `waba-send`, `waba-templates`.

Filas (`src/pages/agente/filas/`): tabela `queues` (ver [docs/chat.md](#)). `useAgentQueues.ts` / `useAgentQueueLimits.ts` (limite por plano). Diálogos: `QueueQRCodeDialog`, `QueueWizardDialog`, `ManageAgentsDialog`, `DisconnectWabaDialog`, `RestoreQueueDialog`, `DeleteQueueDialog`, `QueueAccessDialog`. Edge functions: `queue-provision`, `queue-resolve-phone`, `queue-order-checkout` / `create`, `queue-maintenance`, `sync-queue-to-agent`.

4. Followup — `src/pages/agente/followup/`

Cadências automáticas de reengajamento (Postgres externo, via `externalDb.raw`):

- `followup_config`: `id`, `cod_agent`, `step_cadence/msg_cadence/title_cadence` (jsonb), `start_hours`, `end_hours`, `auto_message`, `followup_from`, `followup_to`. 1 registro/ `cod_agent` (upsert manual). Tratamento defensivo contra double-encoding JSON.
- `followup_queue`: fila de envios por lead — `id`, `cod_agent`, `session_id`, `step_number`, `send_date`, `state`, `history`, `name_client`, `hub`, `chat_memory`. **Estados:** `SEND ("Em FollowUp")` / `STOP ("Parados")`. Query usa `DISTINCT ON (cod_agent, session_id)` ordenado por `send_date DESC` (1 lead = múltiplas linhas históricas, só a mais recente é o estado atual).
- `followup_history`: mensagens efetivamente enviadas (`followup_queue_id` FK, `created_at`).

Métricas: `useFollowupSentCount`, `useFollowupDailyMetrics`, `useFollowupReturnRate`, `useFollowupPreviousPeriodStats`.
Mutations: `useUpdateQueueState`, `useRestartQueueItem`, `useFinalizeQueueItem`.

Parar followup via n8n — `n8n_execute-followup-stop`: recebe `{codAgent, sessionId}`, gera variantes de telefone BR, chama `db-query action followup_stop` (lógica real fica no `db-query`).

5. Contratos — `src/pages/estrategico/contratos/` + `admin/contrato/`

Fonte externa — `JuliaContrato`: `cod_agent`, `agent_id`, `name`, `business_name`, `client_id`, `perfil_agent`, `session_id`, `total_msg`, `whatsapp`, `cod_document`, `zapsing_doctoken`, `status_document`, `situacao`, `data_contrato`, `dataassinatura`, `resumo_do_caso`, `signer_*`, `case_title/category`, `is_confirm`. `JuliaContratoSummary`: `totalContratos`, `contratosAssinados`, `contratosEmCurso`, `taxaAssinatura`.

Assinatura ZapSign: `zapsign-file` (`GET /api/v1/docs/{doc_token}/`, monta ZIP via `jszip`), `zapsign-download`. Link público: https://app.zapsign.com.br/verificar/{zapsign_doctoken}.

Notificações — `contract-notifications-cron` (cron, Postgres externo direto via `postgres client`):

- Fluxo "LEAD_FOLLOWUP": filtra `status_document=== 'CREATED'`, dedupe via `contract_notification_logs` (Supabase), envia via `createMessagingAdapter` (`_shared/messaging-factory.ts`).
- Segundo fluxo: `stepTrigger` `'GENERATED'` (status `CREATED`) ou `'SIGNED'` (status `SIGNED`) — notificações configuráveis por etapa.
- `contract-notifications-queue` — provável enfileiramento de envios individuais.

Template: `admin/contrato/ContratoTemplatePage.tsx`. **Página estratégica:** `ContratosPage.tsx` (`ContratosTable`, `ContratosSummary`, `ContratosEvolutionChart`, `ContratoDetailsDialog`).

6. Integrações Jurídicas

Advbox

`IntegrationPage.tsx` (`useAdvboxIntegration`): seleciona `cod_agent`, token mascarado, testar conexão. `advbox-sync` conecta no Postgres externo direto (`postgresjs` + CA cert); `decryptToken()` com `ADVBOX_ENCRYPTION_KEY` — **token armazenado**

criptografado. Páginas: `ProcessesPage`, `NotificationRulesPage`, `LogsPage`, `QueriesPage`. Edge functions: `advbox-integration`, `advbox-query`, `advbox-notify`.

DataJud (CNJ)

`DataJudSearchPage.tsx`: `SearchBar`, `TribunalSelector`, `ProcessCard`, `MovementTimeline`, `AddProcessDialog`, `BulkImportDialog`, `AlertsPanel`, `MonitoringTab`. Hooks: `useDataJudSearch`, `useMonitoredProcesses`, `useProcessAlerts`, `useNotificationConfig`. `useEnsureDataJudModule.ts` auto-registra o módulo. Edge functions: `datajud-search`, `datajud-monitor` (cron).

Consulta Documento

`consulta-documento` — consulta de documento (CPF/CNPJ ou processo), provável enriquecimento de dados de signatário.

7. Copiloto / IA Admin

Infra de IA compartilhada (`supabase/functions/_shared/aiGateway.ts`):

- Gateway padrão: **Lovable AI Gateway** (`LOVABLE_API_KEY`), fallback opcional **OpenRouter** configurável por feature em `client_ai_model_config` (`feature`, `model`, `provider`, `prompt`).
- Modelos padrão por feature:** `chat_assist`, `chat_resume`, `chat_transcription`, `copilot_crm`, `copilot_chat`, `chat_autoreply`, `support_transcription`, `wavoip_call_summary` → `google/gemini-2.5-flash`; `script_generation` → `google/gemini-3-flash-preview`; `wavoip_transcription` → `openai/gpt-4o-mini-transcribe`. **Nenhum modelo Anthropic/Claude** nesta infra.
- `resolveAI(supabase, feature, fallbackModel)` — chamado por todas as functions de IA, permite trocar modelo/provider sem redeploy.

prompt-generator / `batch-generate-scripts`: `feature` `script_generation` → `google/gemini-3-flash-preview`. Geram prompt/roteiro do agente a partir de caso jurídico + template + config.

Admin de prompts (`src/pages/admin/prompts/`): `PromptsTab`, `TemplatesTab`, `PromptConfigTab`, `GenerateScriptTab`, `LegalCasesTab`, `AgentPromptWizard` (steps: `AgentSearch`→`CaseSelect`→`TemplateSelect`→`AIConfig`→`FinalPrompt`), `DiffViewer` + histórico com diff (prompts/templates/casos).

copilot-chat: `feature` `copilot_chat` → `google/gemini-2.5-flash`; assistente dentro do CRM.

crm-copilot-monitor: `feature` `copilot_crm`; monitora o CRM e gera insights (job periódico) → `admin/copiloto/InsightsMonitorTab.tsx`.

8. Admin

Permissões (`src/pages/admin/permissions/`, **Postgres externo**)

- `AppRole` = `admin|colaborador|user|time|advogado|comercial`. `ModuleCategory` = `principal|crm|agente|sistema|admin|financeiro`.
- `PermissionRow`: `moduleCode`, `moduleName`, `category`, `canView`, `canCreate`, `canEdit`, `canDelete`, `isDefault`.
- `UserWithPermissions`: `id`, `name`, `email`, `role`, `use_custom_permissions`, `is_active`, `parent_user_id`, `remember_token`.
- Hooks: `useUsersWithPermissions`, `useUserPermissions`, `useModules`, `useRoleDefaultPermissions`, `useUpdateUserPermissions`, `useUpdateRoleDefaultPermissions`.
- Componentes: `PermissionMatrix`, `UserPermissionEditor`, `RoleDefaultsDialog`, `UserEditDialog`.

Módulos dinâmicos (`src/pages/admin/modulos/`)

`ModuleFormData`: `code`, `name`, `description?`, `category`, `icon?`, `route?`, `menu_group?`, `is_menu_visible`, `display_order`, `is_active`. Padrão **auto-registro**: dezenas de hooks `useEnsure*Module.ts` (ex.: `useEnsureTicketsModule`, `useEnsureDataJudModule`, `useEnsureCrmComercialModule`) — cada módulo opcional garante sua entrada em `modules` na 1ª montagem, sem migration manual, refletindo no menu (`invalidateQueries(['menu-modules'])`) e na matriz de permissões.

Planos, Pedidos, Meta Ads

- `admin/planos/PlanosPage.tsx`: CRUD de planos comerciais.

- `admin/pedidos/` (`julia_orders` , Supabase, via `useOrders.ts`): `id`, `customer_*`, `plan_name`, `plan_price`, `billing_period`, `status`, `order_nsu`, `checkout_url`, `infinitypay_transaction_nsu`, `receipt_url`, `paid_amount`, `installments`, `webhook_payload`, `cod_agent`, `payment_gateway`, `mp_preference_id`, `mp_payment_id`, `net_amount`, `fee_amount`, `contract_body` . Multi-gateway (InfinityPay + Mercado Pago + Asaas). `OrderDetailSheet.tsx` , `PaymentSettingsDialog.tsx` .
- `admin/meta-ads/` : `MetaAdsTestPage` , `AdAccountSelector` , `CampaignsList` , `ConversionsTest` , `MetaAdsAuth` . Edge functions: `meta-ads` , `meta-auth` , `meta-conversions` , `meta-send-test` , `meta-webhook` .

Dashboard e Estratégico

Dashboard (`src/pages/Dashboard.tsx` , `useDashboardData.ts` , Postgres externo): `DashboardAgent` , `DashboardFiltersState` (`search` , `agentCodes` , `dateFrom` , `dateTo`), `RecentLead` , `DashboardEvolutionData` , `DashboardActivity` , `DashboardStats / _Previous` (`totalLeads` , `totalMessages` , `conversions` , `activeAgents` , `totalSessions` , `mqlCount`), `DashboardFunnelData` . `calculateChange()` calcula variação % vs período anterior. Componentes: `DashboardEvolutionChart` , `DashboardFunnelChart` , `DashboardTripleFunnel` , `DashboardActivityTimeline` .

Estratégico (`src/pages/estrategico/hooks/useJuliaData.ts` , mesma base `vw_painelv2_desempenho_julia*`): `useJuliaAgents` , `useJuliaSesoes(filters)` (filtro `cod_agent` + período + `perfilAgent: SDR|CLOSER|ALL`), `useJuliaSesoesPrevious` , `useJuliaContratos` . Sub-páginas: `DesempenhoPage.tsx` , `CampanhasPage.tsx` (funil por grupo/plataforma, heatmap, top campanhas).

Tickets, Telemetria, Notificações

1. Tickets/Helpdesk

Módulo `/tickets` — chamados de suporte Julia→escritórios (papéis: `requester` / `manager` / `agent` via `useTicketRole`). Ver histórico completo de design em conversas anteriores; abaixo o estado atual do schema/regras.

Tabelas: `support_tickets`, `support_ticket_messages`, `support_departments`, `support_categories`, `support_settings`, `support_ticket_counters`, `support_protocol_counters`.

`support_tickets`: `status` (`open`|`pending`|`in_progress`|`waiting_customer`|`resolved`|`closed`), `priority`, `department_id`, `category_id`, `requester_*`, `requester_client_id`, `assigned_to(name)`, `conversation_id` (vínculo opcional com chat), `contact_id`, `protocol` (numeração amigável), `number`, `opened_at`, `first_response_at`, `resolved_at`, `closed_at`, `reopened_count`, `sla_first_response_due_at`, `sla_resolution_due_at`, `csat_score/comment/at`.

Escopo por papel (`useTickets.ts`): `requester` → `requester_user_id=me`; `manager` / membros de equipe → `requester_client_id` via `resolveEffectiveClientId`; `agent` (`admin`) → todos. Lista padrão exclui `status='closed'`.

Regras de negócio:

- Auto-assign de ticket ao primeiro responsável que interage.
- Protocolo gerado via `sequence/counter`.
- SLA calculado por `department` /prioridade nas `support_settings` (`first_response/resolution`).
- `dispatchToWhatsApp()` : ao responder um ticket vinculado a uma conversa, a mensagem pode ser despachada de volta pro WhatsApp via `queues` (`uazapi/waba`) — resolve fila, contato e envia.
- CSAT ao resolver.
- Notas internas (`support_ticket_messages`) não visíveis ao requester.
- Timeline/eventos junto com mensagens.
- Mídia: `ticket-media-upload`.
- Herança: criar ticket a partir de conversa herda `conversation_id` / `contact_id`; ao trocar de conversa no chat, ticket ativo pode ser fechado automaticamente (`b37067bc "Fechei o ticket ao trocar de conversa"`).

Telas: `TicketsPage.tsx` (Lista/Kanban/Dashboard/Settings por papel, view persistida em `localStorage tickets:lastView`), `TicketDetailPage.tsx`, `TicketsKanban.tsx` (`dnd-kit`, cards por status coloridos), `TicketsListTab.tsx`, `SupportSettingsTab.tsx` (departamentos/categorias/SLA/CSAT), `TicketSlaBadge.tsx`. Integração no chat: `ChatTicketSidePanel.tsx` / `ChatTicketDetailSidePanel.tsx` (ver [docs/chat.md](#)).

2. Telemetria

Captura ambiente (navegador/SO/hardware/rede) e performance (Web Vitals) do usuário. Objetivo: diagnosticar lentidão sem depender de dados exatos de hardware (que o browser não expõe).

Tabelas

- **`user_device_log`** (snapshot por login/sessão): `user_id`, `user_name`, `client_id`, `occurred_at`, `browser(_version)`, `os(_version)`, `device_type`, `cpu_cores`, `device_memory_gb`, `gpu_renderer`, `screen_w/h`, `dpr`, `viewport_w/h`, `net_effective_type`, `net_downlink_mbps`, `net_rtt_ms`, `save_data`, `language`, `timezone`, `user_agent`.
- **`user_performance_log`** (por carregamento/rota): `user_id`, `client_id`, `occurred_at`, `route`, `ttfb_ms`, `fcpx_ms`, `lcp_ms`, `cls`, `dom_interactive_ms`, `load_ms`, `js_heap_used_mb`, `net_effective_type`.
- **View `user_device_latest`**: `DISTINCT ON (user_id) ... ORDER BY user_id, occurred_at DESC` — snapshot mais recente por usuário.
- **`user_activity_log`** (tabela separada, pré-existente): audit trail de login/logout (`event_type`: `login`|`logout_manual`|`logout_inactivity`), com view `user_last_activity`. **Não** é telemetria de device/perf.

RLS permissiva (`USING(true) WITH CHECK(true)`); ambas as tabelas de telemetria na publicação Realtime.

O que é capturado

Ambiente (`src/lib/clientEnvironment.ts`, `collectClientEnvironment()`): navegador/versão e SO via **User-Agent Client Hints** (`navigator.userAgentData.getHighEntropyValues`) com fallback de parse do UA-string; `device_type` (`desktop/mobile/tablet`);

`cpu_cores` = `navigator.hardwareConcurrency`; `device_memory_gb` = `navigator.deviceMemory`; GPU via WebGL UNMASKED_RENDERER_WEBGL; tela/viewport/dpr; rede via `navigator.connection` (effectiveType/downlink/rtt/saveData); idioma/fuso via Intl. Tudo `null` se não suportado (ex.: `deviceMemory` / `connection` são Chromium-only).

Performance (`src/lib/clientPerformance.ts`): `startPerformanceObservers()` registra `PerformanceObserver` para LCP (largest-contentful-paint) e CLS (layout-shift, ignora `hadRecentInput`) uma única vez. `collectPagePerformance()` lê Navigation Timing (TTFB=`responseStart`, `domInteractive`, `loadEventEnd`), Paint Timing (FCP), heap JS (`performance.memory.usedJSHeapSize`, Chromium-only).

Coleta (quando/onde)

- Ambiente: no `login()` (`AuthContext.tsx`) e no restore de sessão já logada (guardado por `sessionStorage` flag para rodar 1x por sessão de browser). Não-bloqueante, falha silenciosa.
- Performance: `usePerformanceReporter()` (`src/hooks/usePerformanceReporter.ts`), montado uma vez em `MainLayout.tsx`. Reporta 2.5s após cada navegação de rota + no `visibilitychange:hidden` (captura LCP/CLS final). Throttle: máx 1 envio/60s por usuário. Só com usuário logado.

Edge function telemetry (`supabase/functions/telemetry/index.ts`)

Actions: `log_device`, `log_performance` (INSERT simples), `get_device_latest` (por lista de `userIds`), `get_user_performance` (últimas 100 amostras de 1 usuário), `get_users_with_telemetry` (lista com último `occurred_at`), `get_dashboard` (agregador pesado):

- Recebe `fromISO` / `bucketMs`. Busca até 20.000 linhas de `user_performance_log` no período + conta sessões (`user_device_log`) + carrega toda `user_device_latest` (composição da frota).
- Calcula: `kpis` (activeNow=usuários c/ amostra nos últimos 5min, sessions, samples, lcpP75/Avg, loadP75/Avg, ttfbAvg, goodRate=%LCP≤2500, weakCount, avgDownlink, avgRtt, avgHeap), `timeseries` (por bucket), `byBrowser/byOs/byDevice/byNetwork` (tallies), `vitals` (good/ni/poor por faixa LCP: ≤2500/2500-4000/>4000), `slowRoutes` (top 8 por LCP médio), `byClient` (top 8 piores `client_id` por LCP p75 + contagem de dispositivos fracos), `recentSlow` (últimas 12 amostras com LCP>2500).
- Heurística "dispositivo fraco" (`isWeak`): `cpu_cores≤2` OU `device_memory_gb≤2` OU `net_effective_type` em `slow-2g/2g/3g`.

Dashboard modo TV — `TelemetryDashboard.tsx`

Em `/admin/monitoramento` → aba "Ambiente & Performance" → sub-aba "Dashboard". Reusa widgets de TV (`BigKpiCard`, `TvCard`, `TvSparklineCard`, `BarRanking` de `src/pages/tv/components/widgets/`). Recursos:

- **Fullscreen** (Fullscreen API nativa) — em fullscreen, tipografia/padding aumentam.
- **Auto-refresh** 10s quando "AO VIVO" ligado (`refreshInterval`); pausável.
- **Seletor de período**: 15m/1h/24h/7d/30d, cada um com seu `bucketMs`.
- KPIs, série temporal (LCP p75 + carga), sparklines, volume de amostras, Core Web Vitals (barra good/NI/poor), saúde de rede, 4 donuts (navegador/SO/dispositivo/rede), rankings (rotas lentas, piores escritórios), tabela de sessões lentas recentes.

Sub-aba "Dados" (`TelemetryExplorer.tsx`): lista de usuários com telemetria (`get_users_with_telemetry`, tempo relativo pt-BR), detalhe por usuário (`get_device_latest` + `get_user_performance` com médias e amostras recentes), aviso de dispositivo fraco.

3. Notificações

- **"Notificar Clientes"** (`/notificar-clientes`, módulo `notify_customers`): notificações internas em tempo real para usuários do sistema.
- `internal_notifications` / `_recipients` (Supabase), edge functions `internal-notification-scheduler` / `internal-notification-dispatch`.
- **Push**: `push_notifications` / `push_subscriptions`, edge function `send-push` (Web Push).
- `NotificationCenter` (componente de UI, sino de notificações).

Telefonia, Pagamentos, Vídeo

1. Telefonia — visão geral

Dois subsistemas independentes + um passivo:

1. **SIP/PBX** (api4com + 3cplus) — softphone clássico com ramais, `PhoneContext` + `useSipPhone` (JsSIP). Tabelas `phone_*`.
2. **Wavoip** — chamadas via WhatsApp usando o SDK `@wavoip/wavoip-webphone`, módulo `/wavoip` independente (`WavoipContext`). Tabelas `wavoip_*`.
3. **vellip** — só webhook passivo de CDR de campanhas, sem discagem.
`queue-*` (filas WhatsApp/WABA) **não são** telefonia-ramal — é sobre canais de chat.

2. Provedores SIP

api4com — `api4com-proxy/index.ts`

Proxy com `action switch`, `Authorization: <api4com_token>`. Actions: `dial` (resolve `phone_extensions.api4com_ramal`), `create_extension` (cria org user + ramal, auto-numeração desde 1000, rollback completo em falha), `update/delete_extension`, `hangup`, `get_sip_credentials` (`{domain, username=ramal, password, wsUrl:wss://{sip_domain}:6443}`, auto-hidrata credenciais faltantes), `sync_extensions`, `sync_call_history` (paginado, `upsert phone_call_logs` por `call_id`), `setup_webhook`, `get_account`.

Webhook `api4com-webhook`: persiste só em `channel-hangup / hangup_cause`; `upsert phone_call_logs`; resolve `cod_agent / client_id` pela extensão; enriquece CDR incompleto via `/calls`.

3cplus — `threecplus-proxy/index.ts`

Auth via `?api_token=`. Actions incluem `get_sip_credentials` (o mais complexo — cadeia de prioridade: **P0** SIP manual (`sip_manual_*`) bypassa 3C+; **P1** login oficial `/agent/webphone/login` com retry em 403; **P2** credenciais cacheadas; **P3 desabilitado** — nunca cai para SIP genérico, retorna `{blocked, errorCode:'WEBPHONE_PERMISSION_REQUIRED'}` se sem licença webphone), `dial` (`/click2call` com token do próprio agente), `hangup`, `create_extension` (`role:agent`, retry de conflito até 20x), `delete_extension`, `sync_extensions`, `sync_call_history`, `diagnose_token`, `validate_sip`.

Webhook `threecplus-webhook`: persiste em `call-was-ended`, resolve agente via `threecplus_agent_id / extension`, `upsert phone_call_logs`.

vellip — `vellip-webhook/index.ts`

Webhook-only. Insere `vellip_call_logs`. Se `cd_resp1=="1"` auto-cria card em `crm_comercial_cards` (stage "Interessados") + `crm_comercial_history`.

3. Softphone (frontend)

`src/pages/telefonია/hooks/useSipPhone.ts` — wrapper JsSIP. Status: `idle|registering|registered|calling|ringing|in-call|error`. WSS auto-discovery, ringtone sintetizado (WebAudio), auto-answer gate (só dentro de janela de 60s após dial do usuário, previne resposta automática a chamada de campanha não solicitada).

`src/contexts/PhoneContext.tsx` — orquestrador: busca extensão ativa do usuário (`phone_extensions.assigned_member_id`), verifica plano ativo (`phone_user_plans`), resolve provider (`phone_config`), auto-conecta SIP com backoff exponencial (máx 8 tentativas, ~5min cap). `dialNumber()` → formata → garante registro SIP → invoca proxy do provider (`getPhoneProxy()` de `src/lib/phoneProxy.ts`) → enfileira `call_id` para sync. Erros não-retryable do 3C+ (falta de licença webphone) viram toast orientando preenchimento manual.

`SoftphoneWidget.tsx` — widget fixo (canto ou overlay central), estados dialing/error/calling/ringing/in-call, teclado DTMF, grace timeout de 15s após encerrar.

Wavoip (softphone alternativo) — `WavoipContext.tsx`: importa dinamicamente `@wavoip/wavoip-webphone`, widget embutido (tema claro forçado). Dispositivos (`wavoip_devices`, compartilháveis via `wavoip_device_members`). `startCall(phoneE164)` normaliza BR E.164, escolhe device conectado, `wp.call.start()`. Eventos SDK atualizam `wavoip_call_logs` + disparam `wavoip-fetch-call-details`. Loop de reconciliação 10s sincroniza status SDK→DB.

4. Provisionamento

SIP — `telephony-provision`

Disparado fire-and-forget pelo `mercadopago-webhook` após pagamento aprovado. Idempotente. Passos: valida `client_id`, escolhe provider default (`telephony_providers.is_default`), insere/reusa `phone_config` (unique por client+provider), desativa planos anteriores + insere novo `phone_user_plans`, marca order `provisioned`. Ramais em si são criados sob demanda via `create_extension`.

Waveip — `wavoip-device-provision (admin)`

Login no provider (`/v2/login`, JWT cacheado), **sempre compra device FREE** (`/v2/sales/buy-device`), insere `wavoip_devices` (`status:'in_use'`, `connection_status:'disconnected'`), renomeia device. `wavoip-connect-device` prepara QR pairing. `wavoip-configure-webhook / verify-webhook` — lifecycle de webhook.

Filas (WhatsApp, não telefonia) — `queue-provision`

Insere `queue_user_plans` (cumulativo) + RPC `apply_queue_limit_from_order` (idempotente) para `chat_client_settings.settings.QUEUE_LIMIT`.

5. Orders/Plans (telefonia + filas + vídeo)

Padrão comum: **create** (valida + insere `draft`, pricing 100% server-side em centavos) → **checkout** (Mercado Pago Checkout Pro — **só MP** para estas 3 famílias). Cliente pode mandar `client_breakdown_cents`; servidor tolera 1 centavo, loga divergência em `metadata`.

- **Telefonia:** `telephony-order-create` (`phone_extension_plans`, add-ons recording/transcription 9990/mês, grátis semestral/anual) → `telephony-order-checkout` (`order_nsu` prefixo TEL-MP-) → `telephony-notify-paid` (WhatsApp de confirmação via UaZapi).
- **Filas:** `queue-order-create / checkout` (`queue_plans`, FIL-MP-).
- **Vídeo:** `video-order-create / checkout` (`video_plans`, add-ons grátis semestral/anual, VID-MP-).

6. Pagamentos — 3 gateways

Todos seguem: `POST {order_id}` → busca order → config em `julia_payment_config` → chama API do provedor → grava `checkout_url + status='pending'`. Webhook flipa para `paid`.

Asaas — `asaas-checkout / webhook`

Auto-detecta sandbox vs produção pelo prefixo da API key. Busca/cria customer por CPF/CNPJ, cria pagamento `CREDIT_CARD` com **12 parcelas fixas**. Webhook processa só `PAYMENT_CONFIRMED / PAYMENT_RECEIVED`, usa `externalReference=order_id`, só atualiza `julia_orders` (**sem provisionamento automático**).

Mercado Pago — `mercadopago-checkout / webhook`

Cria preferência Checkout Pro. `mercadopago-webhook` **é o hub central multi-produto**: resolve tabela de order por `external_reference` tentando `julia_orders` → `queue_orders` → `video_orders` → `telephony_orders`. Em `approved`: seta `paid` + fire-and-forget dispatcha provisionamento (`telephony`→ `telephony-provision` + `telephony-notify-paid`; `queue`→ `queue-provision`; `video`→ `video-provision`).

InfinityPay — `infinitypay-checkout / webhook`

Merchant handle hard-coded. **Sem `external_reference` confiável** — matching por 3 heurísticas: slug do link, `order_nsu`, ou fallback por `plan_price===amount` + mais recente pending (risco de colisão se dois orders tiverem o mesmo preço). Só atualiza `julia_orders`.

Nota de risco: Asaas/InfinityPay não dispõem provisionamento automático (só `julia_orders`, produto SaaS genérico) — só o fluxo Mercado Pago aciona `*-provision` para telefonia/filas/vídeo.

7. Tabelas (payments/orders)

`julia_orders` (SaaS genérico), `julia_payment_config` (config por gateway), `julia_plans`. `queue_orders` + `queue_plans` + `queue_providers`. `video_orders` + `video_plans` + `video_user_plans`. `telephony_orders` + `phone_extension_plans`. Campos comuns entre as 3 famílias de produto: `client_id`, `customer_*`, `plan_id/name`, `billing_period`, `plan_price`, `setup_fee`, `total_amount`, `status`, `payment_gateway`, `order_nsu`, `checkout_url`, `mp_preference_id`, `provisioned_at`, `provisioning_error`, `user_plan_id`, `metadata`.

8. Vídeo — Daily.co

Dois conceitos separados:

- **Compra/assinatura:** `video_orders` → `video_user_plans` (minutos, salas concorrentes, add-ons recording/transcription) — parte do sistema de pagamentos (seção 5/6).
- **Chamada em si:** `video-room/index.ts` (backend Daily.co, `DAILY_API_URL`), tabela `video_call_records`. Actions: `create` (cria sala Daily, 1h expiry, gravação em nuvem, máx 4 participantes), `list` / `lead-waiting` / `queue-status` (fila de espera por `cod_agent`), `operator-join`, `join`, `record-start` / `end`, `close` (finaliza + deleta sala), `history`, `get-recording-link`.
- **Lead-facing:** `JoinCallPage.tsx` (`:roomName`) — estados loading/waiting/ready/error/ended.
- **Operador:** `VideoQueuePage.tsx` — `useVideoRooms`, `useRealtimeQueue`, `useCallHistory`, `useRecordingLink`.

9. Histórico / Gravações / Transcrição (telefonía)

- **SIP:** `phone_call_logs` (`call_id` unique, direction, caller/called, duração, `record_url`, `hangup_cause`). UI: `HistoricoTab.tsx`, `GravacaoPlayer.tsx`.
- **Waveip:** `wavoip_call_logs` (`whatsapp_call_id` unique). Pipeline: `wavoip-call-webhook` (eventos DEVICE/RECORD/CALL) → `wavoip-reconcile-runner` (cron 1min) → `wavoip-reconcile-call` (consulta oficial, dispara download) → `wavoip-fetch-recording` (baixa, sobe pro bucket privado `wavoip-recordings`, URL assinada 5 anos, dispara transcrição). Transcrição (`wavoip-transcribe-recording`) gated por `features.transcription` / `recording_summary` do plano — STT via Lovable AI Gateway (`openai/gpt-4o-mini-transcribe`), resumo opcional via `gemini-2.5-flash`.